

Stack de Monitoring avec Docker — Grafana + Prometheus + Node Exporter

Documentation technique — Mise en place d'une infrastructure de surveillance système sur Ubuntu 24.04 LTS (machine virtuelle)

Table des matières

- 1. [Objectif du projet](#)
- 2. [Installation de Docker](#)
- 3. [Structure des fichiers](#)
- 4. [Configuration des services](#)
- 5. [Lancement de la stack](#)
- 6. [Vérification dans Prometheus](#)
- 7. [Configuration de Grafana](#)
- 8. [Résultat et commandes utiles](#)

1. Objectif du projet

Ce projet a pour but de mettre en place une **infrastructure de surveillance (monitoring) complète** pour une machine virtuelle Linux, en utilisant uniquement des conteneurs Docker. L'objectif est de pouvoir observer **en temps réel** les performances du système : utilisation du processeur, de la mémoire vive, de l'espace disque et du réseau.

Pourquoi surveiller son système ?

Surveiller un système Linux est une pratique fondamentale en administration système et en DevOps. Sans monitoring :

- Il est impossible de détecter une surcharge CPU ou mémoire avant qu'elle ne cause une panne.
- Les problèmes de performance sont difficiles à diagnostiquer après coup.
- Il n'existe aucune visibilité historique sur l'évolution des ressources.

Avec cette stack, vous disposez d'un **tableau de bord interactif** mis à jour toutes les minutes, accessible depuis un simple navigateur web.

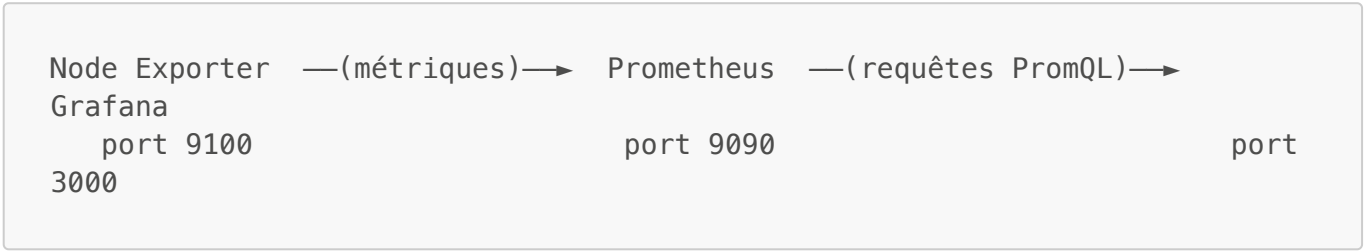
Architecture de la solution

La stack repose sur **trois outils open-source** qui communiquent entre eux dans un réseau Docker isolé :

Composant	Port	Rôle
Node Exporter	9100	S'exécute sur la machine hôte et expose les métriques système (CPU, RAM, disque, réseau) sous un format lisible par Prometheus.

Composant	Port	Rôle
Prometheus	9090	Moteur de collecte et de stockage des métriques. Il interroge (<i>scrape</i>) Node Exporter toutes les 15 secondes et conserve les données dans une base temporelle.
Grafana	3000	Interface de visualisation. Se connecte à Prometheus et permet de construire ou d'importer des tableaux de bord graphiques interactifs.

Le flux de données suit ce chemin :



 Architecture de la stack *Architecture Docker — Node Exporter → Prometheus → Grafana*

Pourquoi Docker ? Utiliser Docker permet de déployer l'ensemble de la stack en une seule commande, sans installer de logiciels directement sur le système hôte. Chaque service est isolé dans son propre conteneur avec ses dépendances, et peut être arrêté ou mis à jour indépendamment.

2. Installation de Docker

Contexte

Docker n'est pas installé par défaut sur Ubuntu 24.04.

⚠ Piège fréquent : La commande `sudo apt-get install docker` n'installe pas Docker sur Ubuntu 24.04 — elle installe **Podman**, une alternative incompatible avec Docker Compose. Il faut utiliser le script d'installation officiel de Docker Inc.

Procédure d'installation

Étape 1 — Télécharger et exécuter le script officiel

Ce script détecte automatiquement la distribution Linux, configure les dépôts Docker officiels et installe la dernière version stable :

```
curl -fsSL https://get.docker.com | sh
```

Étape 2 — Autoriser l'utilisation de Docker sans `sudo`

Par défaut, Docker nécessite des droits root. Pour éviter de préfixer chaque commande avec `sudo`, ajoutez votre utilisateur au groupe `docker` :

```
sudo usermod -aG docker $USER
```

```
sudo usermod -aG docker $USER && newgrp docker
```

`newgrp docker` applique le changement de groupe immédiatement sans nécessiter de déconnexion.

Étape 3 — Vérifier l'installation

```
docker run hello-world
```

Docker télécharge l'image `hello-world` depuis Docker Hub, instancie un conteneur, affiche un message de confirmation, puis le conteneur s'arrête. Si le message s'affiche correctement, l'installation est fonctionnelle.

3. Structure des fichiers

Avant de configurer les services, créez l'arborescence suivante. Cette organisation sépare clairement la configuration de chaque outil :

```
monitoring-stack/  
├── docker-compose.yml          # Orchestration des 3 conteneurs  
├── prometheus/  
│   └── prometheus.yml          # Configuration du scraping  
├── grafana/  
│   └── provisioning/  
│       └── datasources/  
│           └── datasource.yml  # Source de données automatique
```

Pourquoi cette structure ?

- `docker-compose.yml` est le fichier central qui définit comment les conteneurs démarrent et se connectent.
- Les sous-dossiers `prometheus/` et `grafana/` contiennent les fichiers de configuration montés dans les conteneurs via des *volumes*.
- La notion de *provisioning* dans Grafana permet de préconfigurer automatiquement la source de données au démarrage, sans manipulation manuelle dans l'interface web.

Créez les répertoires avec la commande suivante :

```
mkdir -p monitoring-stack/prometheus \  
        monitoring-stack/grafana/provisioning/datasources  
cd monitoring-stack
```

4. Configuration des services

4.1 `docker-compose.yml` — Orchestration des conteneurs

Ce fichier est le cœur de la stack. Il définit les trois services, leurs paramètres de démarrage, les volumes montés et le réseau interne partagé.

```
services:
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
    restart: unless-stopped

  node-exporter:
    image: prom/node-exporter:latest
    container_name: node-exporter
    ports:
      - "9100:9100"
    volumes:
      - /proc:/host/proc:ro
      - /sys:/host/sys:ro
      - /:/rootfs:ro
    command:
      - '--path.procfs=/host/proc'
      - '--path.sysfs=/host/sys'
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin
    volumes:
      - grafana-data:/var/lib/grafana
      - ./grafana/provisioning:/etc/grafana/provisioning
    depends_on:
      - prometheus
    restart: unless-stopped

volumes:
  grafana-data:
```

Explication des points importants :

- **Volumes pour node-exporter** : Les répertoires `/proc`, `/sys` et `/` de la machine hôte sont montés en lecture seule (`:ro`) dans le conteneur. Node Exporter lit ces pseudo-fichiers Linux pour extraire

les métriques système. Sans ces montages, le conteneur ne verrait que ses propres ressources isolées.

- **depends_on** : Grafana attend que Prometheus soit démarré avant de se lancer, évitant les erreurs de connexion au démarrage.
- **restart: unless-stopped** : Les conteneurs redémarrent automatiquement après un crash ou un redémarrage du serveur, sauf si on les arrête manuellement.
- **grafana-data** : Un volume nommé Docker persiste les données Grafana (dashboards, utilisateurs) entre les redémarrages.
- **version obsolète** : L'attribut **version** : en haut du fichier est obsolète dans les versions récentes de Docker Compose. Il peut être supprimé sans impact sur le fonctionnement.

4.2 prometheus/prometheus.yml — Configuration du scraping

Ce fichier indique à Prometheus quelles cibles interroger et à quelle fréquence :

```
global:
  scrape_interval: 15s # Interroge les cibles toutes les 15 secondes

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090'] # Prometheus se surveille lui-même

  - job_name: 'node-exporter'
    static_configs:
      - targets: ['node-exporter:9100'] # Nom du service Docker
```

Résolution DNS interne : Dans un réseau Docker Compose, les conteneurs peuvent se contacter par leur **nom de service**. **node-exporter:9100** est résolu automatiquement en l'adresse IP interne du conteneur **node-exporter**, sans avoir besoin de connaître l'IP réelle.

4.3 grafana/provisioning/datasources/datasource.yml

Ce fichier configure automatiquement Prometheus comme source de données dans Grafana **au démarrage**. Sans ce fichier, il faudrait configurer la source de données manuellement dans l'interface web à chaque recréation des conteneurs.

```
apiVersion: 1

datasources:
  - name: Prometheus
    type: prometheus
```

```
url: http://prometheus:9090
isDefault: true
```

`isDefault: true` indique que Prometheus sera la source de données utilisée par défaut dans tous les nouveaux dashboards Grafana.

 Contenu du fichier `datasource.yml` dans nano *Contenu du fichier `datasource.yml` ouvert dans l'éditeur nano*

5. Lancement de la stack

Démarrage des conteneurs

Une fois les fichiers de configuration en place, lancez l'ensemble de la stack en mode détaché (arrière-plan) :

```
docker compose up -d
```

Le flag `-d` (*detached*) permet aux conteneurs de s'exécuter en arrière-plan sans bloquer le terminal. Au premier lancement, Docker effectue automatiquement les opérations suivantes :

1. **Téléchargement des images** depuis Docker Hub : `prom/node-exporter`, `prom/prometheus`, `grafana/grafana`.
2. **Création du réseau** `monitoring-stack_default` — réseau bridge isolé partagé par les trois conteneurs.
3. **Création du volume** `monitoring-stack_grafana-data` — stockage persistant pour Grafana.
4. **Démarrage des 3 conteneurs** dans l'ordre défini par `depends_on`.

Vérification des conteneurs actifs

Pour vérifier que les trois conteneurs sont bien en cours d'exécution :

```
docker ps
```

 Résultat de `docker ps` — les 3 conteneurs actifs *Résultat de `docker ps` — les 3 conteneurs en statut Up*

Chaque service est maintenant accessible depuis un navigateur :

Service	URL	Usage
Prometheus	<code>http://localhost:9090</code>	Interface de requêtes et vérification des cibles
Node Exporter	<code>http://localhost:9100</code>	Exposition brute des métriques (format texte)
Grafana	<code>http://localhost:3000</code>	Tableau de bord graphique

6. Vérification dans Prometheus

Ouvrez <http://localhost:9090/targets> dans un navigateur. Cette page affiche l'état de toutes les cibles que Prometheus surveille.

Les deux cibles doivent apparaître avec le statut **UP** en vert :

- [node-exporter:9100/metrics](http://localhost:9100/metrics) — les métriques système de la machine hôte
- localhost:9090/metrics — les métriques internes de Prometheus lui-même

En cas de statut DOWN : Vérifiez que le conteneur concerné est bien démarré avec `docker ps`. Consultez les logs avec `docker logs node-exporter` ou `docker logs prometheus` pour identifier le problème.

Si le statut est UP pour les deux cibles, Prometheus collecte correctement les métriques et Grafana peut les exploiter.

7. Configuration de Grafana

Connexion à l'interface

Ouvrez <http://localhost:3000> dans un navigateur. Les identifiants par défaut sont :

Utilisateur	admin
Mot de passe	admin

Grafana demandera de changer le mot de passe lors de la première connexion.

Vérification de la source de données

Grâce au fichier `datasource.yml` configuré précédemment, Prometheus devrait déjà être enregistré automatiquement. Pour le vérifier :

Connections → *Data sources* → sélectionner **Prometheus** → *Save & test*

Si la datasource n'est pas présente, ajoutez-la manuellement :

Add data source → *Prometheus* → URL : <http://prometheus:9090>

⚠ **Important :** Utilisez bien <http://prometheus:9090> (nom du service Docker) et **non** <http://localhost:9090>. Grafana s'exécutant dans un conteneur, `localhost` pointerait vers le conteneur Grafana lui-même et non vers la machine hôte.

Import d'un dashboard communautaire

Grafana met à disposition une bibliothèque de dashboards prêts à l'emploi sur grafana.com/grafana/dashboards. Pour les métriques Linux exposées par Node Exporter, le dashboard le plus utilisé est le **Node Exporter Full** (ID : **1860**).

Procédure d'import :

1. Dans le menu gauche : *Dashboards* → *New* → *Import*
2. Dans le champ *Import via grafana.com*, entrez l'ID : **1860**
3. Cliquez sur **Load**
4. Sélectionnez **Prometheus** comme source de données
5. Cliquez sur **Import**

Le dashboard **Node Exporter Full** affiche en temps réel :

- Utilisation CPU (par mode : user, system, iowait, idle...)
- Consommation RAM (totale, utilisée, cache, swap)
- Débit réseau (entrée/sortie en octets par seconde)
- Utilisation du disque (espace disponible, IOPS)
- Uptime du système

 Dashboard Node Exporter Full *Dashboard Node Exporter Full (ID 1860) — métriques CPU, RAM, réseau et disque en temps réel*

8. Résultat et commandes utiles

Résultat

La stack de monitoring est entièrement opérationnelle. Grafana affiche les performances de la machine virtuelle Linux en temps réel, avec un rafraîchissement automatique configurable (réglé à 1 minute par défaut dans la capture ci-dessus). L'ensemble a été déployé en quelques minutes grâce à Docker Compose, sans aucune installation logicielle directe sur le système hôte.

Commandes de gestion

```
# Démarrer la stack
docker compose up -d

# Arrêter la stack (conteneurs supprimés, données conservées)
docker compose down

# Arrêter ET supprimer les volumes (perte des données Grafana)
docker compose down -v

# Voir les logs en temps réel
docker compose logs -f

# Voir les logs d'un seul service
docker logs grafana
docker logs prometheus
docker logs node-exporter

# Redémarrer un seul service
docker compose restart grafana

# Vérifier le statut des conteneurs
docker ps
```


Pour aller plus loin

D'autres dashboards communautaires sont disponibles sur grafana.com/grafana/dashboards. Quelques suggestions utiles :

ID	Nom	Usage
1860	Node Exporter Full	Métriques système Linux complètes (<i>celui utilisé ici</i>)
893	Docker monitoring	Surveiller les conteneurs eux-mêmes
3662	Prometheus 2.0 Overview	Métriques internes de Prometheus
11074	Node Exporter for ARM	Pour Raspberry Pi et systèmes ARM

Il est également possible d'étendre la stack avec **Alertmanager** pour envoyer des alertes par email ou Slack lorsqu'un seuil est dépassé (ex : CPU > 90% pendant 5 minutes).